Approximation Algorithms - Homework 3

Ilay Menahem, and Aaron Ross

March 12, 2026

Problem 1

i. Proof of Lemma 2

Lemma 2. *One can determine in polynomial time if there exists a schedule of length T for the rounded long jobs.*

Proof. Let k be a fixed constant. Recall that a job j is defined as "long" if its processing time $p_j > T/k$. The scheme $\{B_k\}$ rounds each long job processing time p_j down to p'_j, where p'_j is the nearest multiple of T/k^2.

1. Bounding the Number of Job Types: Since $T/k < p_j \leq T$ for all long jobs, the possible values for p'_j are restricted to the set:

$$\mathcal{V} = \left\{ i \cdot \frac{T}{k^2} \mid i \in \{k, k+1, \ldots, k^2\} \right\}$$

The number of distinct job sizes (types) is $d = |\mathcal{V}| = k^2 - k + 1$. Crucially, d is a constant that depends only on k and not on the input size n.

2. Machine Configurations: A "configuration" is a vector $(s_1, s_2, \ldots, s_d)$ where s_i is the number of jobs of size v_i assigned to a single machine. A configuration is *valid* if:

$$\sum_{i=1}^{d} s_i \cdot v_i \leq T$$

Because each $v_i > T/k$, a machine can process at most k long jobs. Thus, $\sum s_i \leq k$. The total number of valid configurations Q is at most $(k+1)^d$. Since k and d are constants, Q is also a constant.

3. Dynamic Programming Formulation: Let the set of distinct rounded job sizes be $\mathcal{V} = \{v_1, v_2, \ldots, v_d\}$. We define a state vector $\mathbf{n} = (n_1, n_2, \ldots, n_d)$, where each component n_i represents the number of jobs of size v_i to be scheduled.

Let $W(\mathbf{n})$ denote the minimum number of machines of capacity T required to schedule the multiset of jobs defined by $\mathbf{n}$. The recurrence relation is:

$$W(\mathbf{n}) = 1 + \min_{\mathbf{s} \in \mathcal{C}} \{W(\mathbf{n} - \mathbf{s})\}$$

where $\mathcal{C}$ is the set of all valid machine configurations $\mathbf{s} = (s_1, \ldots, s_d)$ such that $\sum_{i=1}^{d} s_i v_i \leq T$ and $s_i \leq n_i$ for all $i \in \{1, \ldots, d\}$.

Correctness of the DP Formulation

The correctness of the DP relies on the principle of optimality. Let $\mathbf{n}$ be the vector of job counts. We will prove by induction on $\sum_{n_i \in n} n_i$.

Proof by Induction: **Base Case:** $W(\mathbf{0}) = 0$. If no jobs are to be scheduled, zero machines are required, which is optimal.

Inductive Step: Assume $W(\mathbf{n}')$ is the minimum number of machines required for all vectors $\mathbf{n}'$ such that $\sum n'_i < \sum n_i$. To schedule the jobs in $\mathbf{n}$, we must assign a non-empty subset of jobs $\mathbf{s}$ to at least one machine. For any valid configuration $\mathbf{s} \in \mathcal{C}$, the remaining jobs are $\mathbf{n} - \mathbf{s}$. By the inductive hypothesis, $W(\mathbf{n} - \mathbf{s})$ is the optimal number of machines for the remaining jobs. The total number of machines used for configuration $\mathbf{s}$ is $1 + W(\mathbf{n} - \mathbf{s})$. To minimize the total machines, we must minimize over all possible configurations for that machine:

$$W(\mathbf{n}) = \min_{\mathbf{s} \in \mathcal{C}} \{1 + W(\mathbf{n} - \mathbf{s})\}$$

Since the number of distinct configurations $|\mathcal{C}|$ is finite and exhaustive for the capacity T, the recurrence correctly identifies the global minimum.

4. Complexity: State Space Size: The DP table has one entry for every possible vector $\mathbf{n}$. Since each n_i can range from 0 to n (the total number of jobs), there are $O(n^d)$ possible states.
Transition Cost: For each state, we iterate over the set C, which represents all valid machine configurations. The time to fill the table is $O(Q \cdot n^d)$. Substituting $d = O(k^2)$, the complexity is $O(n^{O(k^2)})$. Since k is a constant, this is polynomial in n. $\square$

ii. Total Running Time of the $\{B_k\}$ Scheme

Proof. To prove that the total running time of $\{B_k\}$ is polynomial in the input size, we analyze the three main components of the algorithm: the binary search for T, the dynamic programming for long jobs, and the greedy placement of short jobs.

1. Binary Search for T: The algorithm searches for the minimum T such that the scheme $\{B_k\}$ returns a valid schedule.

- Let B be the size of the input in bits

- Let $L = \max\{\max_j p_j, \frac{1}{m}\sum p_j\}$ and $U = \sum p_j$. The optimal makespan $C^*_{\max}$ must lie in $[L, U]$.
- We perform a binary search over this interval. To ensure the final $(1+\epsilon)$ approximation ratio, the search terminates when the range is reduced to a precision $\delta = \text{poly}(\epsilon, n)$.
- The number of iterations is $\lceil \log_2(\frac{U-L}{\delta}) \rceil$. Since U and L are represented in the input using B bits, $\log_2(U)$ is $O(B)$. Thus, the number of iterations is linear in the size of the input (bits), which is polynomial.

2. Complexity per Iteration (Steps 2-4): For each T chosen by the binary search:

- **Partitioning (Step 2):** Splitting n jobs into long and short sets takes $O(n)$.
- **Rounding (Step 3):** Rounding the n jobs takes $O(n)$.
- **Solving for Long Jobs (Step 4):** As proven in Lemma 2, the DP takes $O(Q \cdot n^d)$ where $Q \leq (k+1)^{k^2}$ and $d = k^2 - k + 1$. Since $k = 1/\epsilon$ is a constant, this is $O(n^{O(k^2)})$, which is polynomial in n.

3. Scheduling Short Jobs (Step 5): Once the long jobs are scheduled, the algorithm places the short jobs using the LS approach shown in class, which takes $O(n \cdot m)$

4. Conclusion: The total time complexity is the product of the binary search iterations and the work done per iteration:

$$\text{Total Time} = O\left(\text{B} \cdot (n + n^{O(k^2)} + n \cdot m)\right)$$

Since k is a constant, the term $n^{O(k^2)}$ dominates, but it remains a polynomial function of n. Therefore, the total running time of the $\{B_k\}$ scheme is polynomial in the input size. $\square$

Proof of Lemma 1

Lemma 1. *Given the variables C_j, if the constraints in (2) are satisfied for the n prefix subsets $S_1, S_2, \ldots, S_n$ (where $S_k = \{1, \ldots, k\}$ and $C_1 \leq C_2 \leq \cdots \leq C_n$), then the constraints are satisfied for any subset $S \subseteq N$.*

Proof. Assume towards contradiction that the constraints in (2) are satisfied for all prefix sets $S_1, \ldots, S_n$, but there exists a subset $A \subseteq N$ that violates the constraint:

$$\sum_{j \in A} p_j C_j < \frac{p(A)^2}{2} \tag{1}$$

where $p(A) = \sum_{j \in A} p_j$. We define a process that transforms A into one of the prefix sets while preserving the violation. Let $m = \max A$. We consider two exhaustive cases:

Case 1: Removing the maximum element

If $p_m C_m \geq \sum_{j \in A \setminus \{m\}} p_m p_j + \frac{p_m^2}{2}$, we show that $A \setminus \{m\}$ also violates (2). By definition:

$$\begin{aligned}
\frac{p(A \setminus \{m\})^2}{2} &= \frac{(p(A) - p_m)^2}{2} \\
&= \frac{p(A)^2}{2} - p(A)p_m + \frac{p_m^2}{2} \\
&= \frac{p(A)^2}{2} - \left(\sum_{j \in A \setminus \{m\}} p_m p_j + \frac{p_m^2}{2} \right)
\end{aligned}$$

Subtracting $p_m C_m$ from both sides of the violation inequality (1)

$$\begin{aligned}
\sum_{j \in A \setminus \{m\}} p_j C_j &= \sum_{j \in A} p_j C_j - p_m C_m \\
&< \frac{p(A)^2}{2} - \left(\sum_{j \in A \setminus \{m\}} p_m p_j + \frac{p_m^2}{2} \right) = \frac{p(A \setminus \{m\})^2}{2}
\end{aligned}$$

Thus, $A \setminus \{m\}$ violates the constraint.

Case 2: Adding a missing smaller element

If $p_m C_m < \sum_{j \in A \setminus \{m\}} p_m p_j + \frac{p_m^2}{2}$, then for any $i \in [m] \setminus A$, we show that $A \cup \{i\}$ violates (2). Since $C_i \leq C_m$ and dividing the case assumption by p_m gives $C_m < \sum_{j \in A \setminus \{m\}} p_j + \frac{p_m}{2}$, we have:

$$p_i C_i \leq p_i C_m < p_i \left(\sum_{j \in A \setminus m} p_j + \frac{p_m}{2} \right) \quad = \sum_{j \in A \setminus m} p_i p_j + \frac{p_i p_m}{2} \quad < \sum_{j \in A \setminus m} p_i p_j + (p_i p_m) + \frac{p_i^2}{2} \quad = \sum_{j \in A} p_i p_j + \frac{p_i^2}{2}$$

(Since $p_i, p_m, p_i^2 > 0$)

Adding this to the violation inequality (1)

$$\begin{aligned}\sum_{j\in A\cup\{i\}} p_jC_j &= \sum_{j\in A} p_jC_j + p_iC_i \\ &< \frac{p(A)^2}{2} + \sum_{j\in A} p_ip_j + \frac{p_i^2}{2} = \frac{p(A\cup\{i\})^2}{2}\end{aligned}$$

Thus, $A \cup \{i\}$ violates the constraint.

Transformation and Contradiction

We define a procdeure which, while A is not a prefix set, either removes the maximum element (Case 1) or adds a missing element $i < \max A$ (Case 2). We observe that each iteration of the transformation maintains the invariant that the current set A violates constraint (2). To ensure termination, note that each job index $j \in \{1, \ldots, n\}$ is either added to or removed from A at most once. Specifically, removing the maximum element $m = \max(A)$ precludes the re-insertion of m or any index $j > m$ into the set. Since there are at most n insertions and n removals, the procedure terminates in $O(n)$ steps.

Upon termination, A cannot be further modified, which occurs if and only if A contains no "gaps" below its maximum element, implying A is a prefix set $S_k = \{1, \ldots, k\}$. By our invariant, this terminal set S_k must violate (2), which contradicts the hypothesis that all n prefix sets satisfy the constraint. Thus, no such violating set A can exist.

□

Problem 2

(a) *Proof.* We begin by formulating the linear programming relaxation for the Set Cover problem (LP_{SC}):

$$\begin{aligned} \text{Minimize} \quad & \sum_{j=1}^{m} w_j x_j \\ \text{Subject to} \quad & \sum_{j: e_i \in S_j} x_j \geq 1 \quad \forall i \in [n] \\ & x_j \geq 0 \quad \forall j \in [m] \end{aligned}$$

Let OPT be the objective value of the optimal solution to this LP. We construct the corresponding dual linear program. By introducing a dual variable $y_i \geq 0$ for each element $e_i \in [n]$, we obtain:

$$\begin{aligned} \text{Maximize} \quad & \sum_{i=1}^{n} y_i \\ \text{Subject to} \quad & \sum_{i: e_i \in S_j} y_i \leq w_j \quad \forall j \in [m] \\ & y_i \geq 0 \quad \forall i \in [n] \end{aligned}$$

Let x^* and y^* be optimal solutions to the primal and dual LPs, respectively. According to the Complementary Slackness theorem, if $x_j^* > 0$, then the corresponding dual constraint must be tight. That is, for every $j \in [m]$, if $x_j^* > 0$, then $\sum_{i: e_i \in S_j} y_i^* = w_j$.

Our modified algorithm constructs a solution by selecting exactly the sets $A = \{j : x_j^* > 0\}$. The total cost of this generated solution is:

$$\sum_{j \in A} w_j = \sum_{j \in A} \left(\sum_{i: e_i \in S_j} y_i^* \right)$$

We can change the order of summation by counting how many times each dual variable y_i^* appears. The variable y_i^* is included exactly once for every chosen set $S_j \in A$ that contains e_i. Thus:

$$\begin{aligned} \sum_{j \in A} w_j &= \sum_{i=1}^{n} y_i^* \cdot |\{j \in A : e_i \in S_j\}| \\ &\leq \sum_{i=1}^{n} y_i^* \cdot f_i \qquad \text{(Where } f_i \text{ is the frequency of } e_i \text{ in the input)} \\ &\leq f \sum_{i=1}^{n} y_i^* \end{aligned}$$

By Strong Duality, the value of the optimal dual solution equals the value of the optimal primal solution, meaning $\sum_{i=1}^{n} y_i^* = OPT$. Therefore, the cost of our solution is at most

$f \cdot OPT$. Since solving the LP and selecting the sets takes polynomial time, this is a valid polynomial-time f-approximation algorithm. $\square$

(b) *Proof.* Given a Vertex Cover (VC) instance on a graph $G = (V, E)$, we can naturally map it to a Set Cover (SC) instance. Let the universe of elements be the edges $E = \{e_1, \ldots, e_n\}$. For each vertex $v_j \in V$, we define a set $S_j = \{e_i \in E : v_j \in e_i\}$ containing all edges incident to v_j. We assign a weight of 1 to every set.

In the lecture we saw that this problem is equivalent to the original VC problem. Therefore, because every edge in a standard graph(no self loops) connects exactly two vertices, each element in our SC instance appears in exactly 2 sets. Therefore, the maximum frequency of any element is $f = 2$.

Applying the f-approximation algorithm from part (a) to this SC instance will yield a solution with a cost of at most $2 \cdot OPT$. The transformation requires $O(|V| + |E|)$ time, which is polynomial, additionally getting the solution for vertex cover from the set cover solution can be done in polynomial time by taking the vertices that represent the sets in the solution. making this a valid 2-approximation for Vertex Cover. $\square$

(c) Let $y_i \in \{0, 1\}$ be an indicator variable representing the truth value assigned to x_i, where $y_i = 1$ implies True and $y_i = 0$ implies False.

For any literal z, we define its algebraic value $v(z)$ as follows:

- If $z = x_i$, then $v(z) = y_i$.
- If $z = \neg x_i$, then $v(z) = 1 - y_i$.

To satisfy a clause $c_j = z_{j1} \vee z_{j2}$, at least one literal must be True, which enforces the linear constraint $v(z_{j1}) + v(z_{j2}) \geq 1$. The goal is to minimize the number of True variables. The Integer Linear Program (ILP) is:

$$\begin{aligned} \text{Minimize} \quad & \sum_{i=1}^{n} y_i \\ \text{Subject to} \quad & v(z_{j1}) + v(z_{j2}) \geq 1 \quad \forall j \in [m] \\ & y_i \in \{0, 1\} \quad \forall i \in [n] \end{aligned}$$

(d) *Proof.* We first check if the given 2-SAT instance is satisfiable in polynomial time (since 2-SAT $\in$ P). If it is unsatisfiable, we terminate. Otherwise, we relax the ILP to a continuous Linear Program by replacing the integrality constraint with $0 \leq y_i \leq 1$ for all $i \in [n]$.

Let y^* be the optimal fractional solution to this LP. We apply the following rounding algorithm to produce an assignment $v(x_i)$:

(a) If $y_i^* > \frac{1}{2}$, set $v(x_i) = 1$.

(b) If $y_i^* < \frac{1}{2}$, set $v(x_i) = 0$.

(c) If $y_i^* = \frac{1}{2}$, leave x_i temporarily unassigned.

Lemma 2. *The residual 2-SAT formula, formed by simplifying the clauses with the deterministically assigned variables, remains satisfiable.*

Proof of Lemma. Assume for contradiction that the residual formula is unsatisfiable. This implies that the partial assignment falsified a clause entirely, meaning both of its literals were assigned 0. For a literal to evaluate to 0 in our rounding scheme, its corresponding LP value must have been strictly less than $\frac{1}{2}$. Thus, for the falsified clause $c_j = z_{j1} \vee z_{j2}$, we must have had $v(z_{j1})^* < \frac{1}{2}$ and $v(z_{j2})^* < \frac{1}{2}$.

Summing these gives $v(z_{j1})^* + v(z_{j2})^* < 1$, which strictly violates the LP constraint for clause c_j. This is a contradiction, confirming the remaining formula is satisfiable. □

Because the residual problem is a valid, satisfiable 2-SAT instance, we can use a standard polynomial-time 2-SAT solver to find a satisfying assignment for the remaining variables (those where $y_i^* = \frac{1}{2}$).

To establish the approximation ratio, observe the relationship between our final assignment $v(x_i)$ and the optimal LP values y_i^*:

- If $y_i^* > \frac{1}{2}$, we set $v(x_i) = 1$, which respects $v(x_i) < 2y_i^*$.
- If $y_i^* < \frac{1}{2}$, we set $v(x_i) = 0 \leq 2y_i^*$.
- If $y_i^* = \frac{1}{2}$, $v(x_i)$ becomes either 0 or 1. In both cases, $v(x_i) \leq 1 = 2(\frac{1}{2}) = 2y_i^*$.

Therefore, $v(x_i) \leq 2y_i^*$ holds for all variables. The total weight of our assignment is:

$$\sum_{i=1}^{n} v(x_i) \leq \sum_{i=1}^{n} 2y_i^* = 2 \cdot OPT_{LP} \leq 2 \cdot OPT$$

This guarantees a 2-approximation. Since LP solving, partial rounding, and 2-SAT solving all take polynomial time, the entire algorithm is efficient. □

Problem 3

Part (a):

To prove that the given Linear Program is a relaxation of the FACILITY LOCATION problem, we will encode the problem as an IP and show that it is identical to the given LP but with relaxed constraints.

IP formulation

In the discrete Facility Location problem, we are asked to find a subset of open facilities $S \subseteq F$ that minimizes the total opening and connection costs:

$$\min \left(\sum_{i \in S} f_i + \sum_{j \in C} d(j, S) \right)$$

To formulate this as an optimization problem, we introduce binary indicators to represent both facility opening and client assignment decisions across the set F

- **Facility opening variables:** Let $y_i \in \{0, 1\}$ indicate whether facility $i \in F$ is chosen to be opened ($y_i = 1$) or remains closed ($y_i = 0$).
- **Client assignment variables:** Let $x_{ij} \in \{0, 1\}$ indicate whether client $j \in C$ is assigned to facility $i \in F$ ($x_{ij} = 1$) or not ($x_{ij} = 0$).

Using these variables, we can formulate the objective function and add the necessary constraints to ensure a valid assignment:

1. **The Objective Function:**
   We replace the set S by summing over all facilities in F and multiplying by our indicator variables. We pay f_i if $y_i = 1$, and we pay the distance d_{ij} if $x_{ij} = 1$. The objective becomes:
   $$\min \sum_{i \in F} f_i y_i + \sum_{j \in C} \sum_{i \in F} d_{ij} x_{ij}$$
2. **The Assignment Constraint:**
   We must count exactly one distance from each client to a facility (meaning every client must be served by exactly one facility). Therefore, for every client j, the sum of their assignment variables must equal 1:
   $$\sum_{i \in F} x_{ij} = 1 \quad \forall j \in C$$
3. **The Open Facility Constraint:**
   We cannot assign a client to a facility that is closed. Therefore, if we use the distance from client j to facility i (meaning $x_{ij} = 1$), it forces the facility to be open (y_i must be 1). This is enforced by bounding the assignment variable by the opening variable:
   $$x_{ij} \leq y_i \quad \forall i \in F, j \in C$$

Final IP: Combining these elements, the exact Integer Program for the Facility Location problem is:

$$\begin{aligned}
\min \quad & \sum_{i \in F} f_i y_i + \sum_{j \in C} \sum_{i \in F} d_{ij} x_{ij} \\
\text{s.t.} \quad & \sum_{i \in F} x_{ij} = 1, \quad \forall j \in C \\
& x_{ij} \leq y_i, \quad \forall i \in F, j \in C \\
& x_{ij}, y_i \in \{0, 1\}, \quad \forall i \in F, j \in C
\end{aligned}$$

Relaxation to LP: The LP provided in the problem is the *linear relaxation* of this IP. By relaxing the integer constraints $x_{ij}, y_i \in \{0, 1\}$ to the continuous box constraints $x_{ij}, y_i \in [0, 1]$, we expand the feasible region.

Conclusion: Let S_{IP} be the set of integer-feasible points and S_{LP} be the set of fractional-feasible points. Since $S_{IP} \subset S_{LP}$, it follows that:

$$Z^*_{LP} = \min_{(x,y) \in S_{LP}} Z(x, y) \leq \min_{(x,y) \in S_{IP}} Z(x, y) = Z^*_{IP}$$

Thus, the LP optimal value is a lower bound on the optimal integer value, confirming the LP is a valid relaxation.

Part (b):

Let (x, y) be an optimal fractional solution to the LP, Let $C_j = \sum_{i \in F} d_{ij} x_{ij}$. We define the set of "close" facilities as $F_j = \{i \in F : d_{ij} \leq \frac{4}{3} C_j\}$.

1. Proving Inequality (1): $\sum_{i \in F} \tilde{x}_{ij} \geq 1$

Proof. Suppose, for the sake of contradiction, that $\sum_{i \in F} \tilde{x}_{ij} < 1$, thus

$$\sum_{i \in F} \tilde{x}_{ij} < 1 \to \sum_{i \in F_j} 4x_{ij} < 1 \to \sum_{i \in F_j} x_{ij} < \frac{1}{4}$$

. Since the LP constraint (1) ensures $\sum_{i \in F} x_{ij} \geq 1$, it must be that $\sum_{i \notin F_j} x_{ij} > \frac{3}{4}$. By definition, for any $i \notin F_j$, we have $d_{ij} > \frac{4}{3} C_j$. Then:

$$C_j = \sum_{i \in F} d_{ij} x_{ij} \geq \sum_{i \notin F_j} d_{ij} x_{ij} > \sum_{i \notin F_j} \left(\frac{4}{3} C_j \right) x_{ij} = \frac{4}{3} C_j \sum_{i \notin F_j} x_{ij}$$

Substituting our assumption $\sum_{i \notin F_j} x_{ij} > \frac{3}{4}$:

$$C_j > \frac{4}{3} C_j \left(\frac{3}{4} \right) = C_j$$

We get $C_j > C_j$, which is a contradiction Therefore, inequality (1) is satisfied. □

2. Disproving Inequality (2): $y_i \geq \tilde{x}_{ij} \geq 0$

The solution $(\tilde{x}, y)$ **does not** necessarily satisfy $y_i \geq \tilde{x}_{ij}$.

Proof. Consider a client j and 4 facilities where $d_{ij} = 1$ and $f_i = 1$ for all i. Let the LP solution be $y_i = 0.25$ and $x_{ij} = 0.25$ for $i \in \{1, 2, 3, 4\}$. It is easy to see that this is an optimal solution for the LP. Here, $C_j = 1$, so all facilities are "close" ($\leq \frac{4}{3}$). Then $\tilde{x}_{ij} = 4x_{ij} = 4(0.25) = 1$. However, $y_i = 0.25$. Since $0.25 < 1$, the constraint $y_i \geq \tilde{x}_{ij}$ is violated. $\square$

Part (c):

Let (x, y) be an optimal solution to the LP relaxation. For each client $j \in C$, let $C_j = \sum_{i \in F} d_{ij} x_{ij}$ be its fractional connection cost. We define the set of facilities "close" to client j as $F_j = \{i \in F : d_{ij} \leq \frac{4}{3} C_j\}$.

The Algorithm

1. Let $U = C$ be the set of unassigned clients, and let $S = \emptyset$ be the set of opened facilities.
2. Solve the LP and obtain and optimal solution (x, y).
3. For each client $j \in C$, calculate $C_j = \sum_{i \in F} d_{ij} x_{ij}$ and $F_j = \{i \in F : d_{ij} \leq \frac{4}{3} C_j\}$
4. While $U \neq \emptyset$:
   (a) Select a "center" client $j \in U$ that minimizes C_j.
   (b) Open the facility $i^* \in F_j$ that minimizes the opening cost f_i. Add i^* to S.
   (c) Assign client j and all currently unassigned clients $k \in U$ such that $F_k \cap F_j \neq \emptyset$ to the facility i^*.
   (d) Remove all clients assigned in the previous step from U.
5. return S

The algorithm returns a solution $S \neq \emptyset$ and $S \subseteq F$, therefore, it's a valid solution to the Facility Location problem.

2. Proof of 4-Approximation

We will bound the total facility opening cost and the total connection cost relative to the optimal LP objective value, $OPT_{LP} = \sum_{i \in F} f_i y_i + \sum_{i \in F} d_{ij} x_{ij} = \sum_{i \in F} f_i y_i + \sum_{j \in C} C_j$.

Bounding the Facility Opening Cost

From part(b) we can conclude that the following inequality holds

$$\sum_{i \in F} \tilde{x}_{ij} \geq 1 \rightarrow \sum_{i \in F_j} 4x_{ij} \geq 1 \rightarrow \sum_{i \in F_j} x_{ij} \geq \frac{1}{4}$$

. Since the LP constraints require $y_i \geq x_{ij}$ for all i, j, it follows that $\sum_{i \in F_j} y_i \geq 1/4$.

When the algorithm processes client j and opens $i^* \in F_j$, it chooses the absolute cheapest facility in F_j. The cost f_{i^*} can be bounded by the weighted average cost of facilities in F_j:

$$f_{i^*} \leq \frac{\sum_{i \in F_j} f_i y_i}{\sum_{i \in F_j} y_i} \leq \frac{\sum_{i \in F_j} f_i y_i}{1/4} = 4 \sum_{i \in F_j} f_i y_i$$

Let J be the set of "center" clients chosen in step 4(a). By the removal condition in step 3(d), for any two distinct center clients $j, j' \in J$, their close facility sets are completely

disjoint ($F_j \cap F_{j'} = \emptyset$). Because these sets are disjoint, we can safely sum their costs without overcounting any fractional y_i mass:

$$\sum_{i \in S_{alg}} f_i = \sum_{j \in J} f_{i^*(j)} \leq \sum_{j \in J} 4 \sum_{i \in F_j} f_i y_i \leq 4 \sum_{i \in F} f_i y_i$$

where $f_{i^*(j)}$ is the facility assigned to client j, and the first equality holds because facilities are only opened by center clients(clients chosen in step 4(a)).

Bounding the Connection Cost

Consider any client $k \in C$ assigned to a facility i^* opened by some center client $j \in J$. There are two possible cases:

- **Case 1: $k = j$ (The center client itself).**
  Client j is assigned to $i^* \in F_j$. By the definition of the set F_j, the distance is bounded by:
  $$d(j, i^*) \leq \frac{4}{3} C_j \leq 4C_j$$

- **Case 2: $k \neq j$ (A client clustered with j).**
  Since k was assigned along with j, we know $F_k \cap F_j \neq \emptyset$. Let i' be a facility in this intersection. By the triangle inequality:
  $$d(k, i^*) \leq d(k, i') + d(i', j) + d(j, i^*)$$
  Since $i' \in F_k$, we have $d(k, i') \leq \frac{4}{3} C_k$.
  Since $i' \in F_j$ and $i^* \in F_j$, we have $d(i', j) \leq \frac{4}{3} C_j$ and $d(j, i^*) \leq \frac{4}{3} C_j$.
  Substituting these values gives:
  $$d(k, i^*) \leq \frac{4}{3} C_k + \frac{4}{3} C_j + \frac{4}{3} C_j = \frac{4}{3} C_k + \frac{8}{3} C_j$$
  Because the algorithm always selects the unassigned client with the minimum C_j to be the next center, and k was still unassigned when j was chosen, it must be true that $C_j \leq C_k$. Thus:
  $$d(k, i^*) \leq \frac{4}{3} C_k + \frac{8}{3} C_k = \frac{12}{3} C_k = 4C_k$$

In all cases, the connection cost for any client k is at most $4C_k$. Summing over all clients yields:

$$\sum_{k \in C} d(k, i^*(k)) \leq 4 \sum_{k \in C} C_k$$

where $i^*(k)$ is the facility assigned to client k in the algorithm

Conclusion

Adding the bounded facility opening costs and connection costs together, the total cost of our integer solution is:

$$\text{Total Cost} \leq 4\sum_{i\in F} f_i y_i + 4\sum_{k\in C} C_k = 4\left(\sum_{i\in F} f_i y_i + \sum_{k\in C} C_k\right) = 4 \cdot OPT_{LP} \leq 4 \cdot OPT$$

Since an optimal solution to the LP is a lower bound on the Facility Location problem(shown in part(a)).

Complexity Analysis:

The LP runs in $O(poly(n))$ time. The calculation of C_j, F_j can be done in $O(n)$ time for each j, Therefore a total of $O(n^2)$ for all clients. The loop has at most $|C|$ iterations and each iteration can be done trivially in $O(n^2)$ time. Therefore, the loop is bound by $O(n^3)$. In total the runtime is $max(poly(n), O(n^3))$ where $poly(n)$ is the runtime of the LP

Problem 4: asymptotic PTAS for Bin Packing with trees

The problem

In the Bin Packing with Trees problem we are given a collection of $n > 0$ items $I = \{1, 2, \ldots, n\}$, each item has a size s_i, $0 < s_i \leq 1$. Also, we are given an undirected tree $T = (V, E)$. That is, the set of vertices of T is the set of items (i.e., $V = I$). A feasible solution for the problem is an m-tuple $U = (U_1, U_2, \ldots, U_m)$ for some $m \geq 1$, such that

- For every j, $1 \leq j \leq m$, $U_j \subseteq I$ and $\sum_{i \in U_j} s_i \leq 1$.
- For every $i \in I$ there is $j \in [m]$ such that $i \in U_j$.
- For all $j \in [m]$ and $\{u, v\} \in E$ it holds that $|U_j \cap \{u, v\}| \leq 1$. That is, two items connected by an edge cannot be packed in the same bin.

An asymptotic PTAS (APTAS) for a minimization problem is a family of algorithms $\{A_\epsilon\}$ and a constant $c > 0$, such that there is an algorithm A_ϵ for any $\epsilon > 0$, and A_ϵ outputs a solution of value at most $(1 + \epsilon)\mathrm{OPT} + c$, where OPT is the value of an optimal solution. Suggest an APTAS for Bin Packing with Trees.

The algorithm

Solution outline - First we will divide the items into large and small ones, the large ones into intervals of sizes, and we will round the sizes of the items in each interval to the maximum size in the interval. Then we will use the standard APTAS to get an APTAS for the abstracted sizes, we will resolve the conflicts, and at last we will add the small items to the bins.

Conflict resolution

Now we will describe a conflict resolution algorithm for a tree conflict graph. The input *AbstractedBins* is the solution produced by standard APTAS on the rounded instance: a list of bins, where each bin is a multiset of abstracted sizes (one slot per large item assigned to it). The goal is to assign each real item $i \in Large$ to a slot whose abstracted size equals i's abstracted size, such that no two adjacent items share a bin.

The idea is to order the vertices so that at most one neighbor precedes each vertex. Given such an order, we greedily assign each vertex to an available slot of its abstracted size in a bin that contains no already-assigned neighbor. If no such bin exists, we open a new bin.

Here is an algorithm that finds the desired order of the vertices in the tree:

Algorithm 1 1-inductive tree ordering

```
procedure OneInductiveTreeOrdering(T = (V, E))
    Pick any vertex r ∈ V as the root
    return BFS(T, r)                    ▷ vertices in BFS discovery order
end procedure
```

Claim 1. *The order returned by the algorithm is a 1-inductive order.*

Proof. Since T is a tree (acyclic), the BFS spanning tree is T itself. Every non-root vertex v has exactly one parent, the vertex that discovered it, which precedes v in BFS order. Every other neighbor of v is a child of v in the BFS tree, meaning v discovered it, so it appears *after* v. The root has no preceding neighbors at all. Hence every vertex has at most one preceding neighbor. □

Algorithm 2 Conflict resolution for Bin Packing with Trees

```
procedure ConflictResolution(AbstractedBins, T)  ▷ AbstractedBins: list of bins with slots of abstracted sizes
    Order ← OneInductiveTreeOrdering(T)
    ResolvedBins ← []
    for each vertex v in Order do
        ŝ_v ← abstracted size of v
        Let S be the set of bins in AbstractedBins with an unmatched slot of size ŝ_v
        Filter S to only include bins that do not contain any already-matched neighbor of v
        if S is not empty then
            Pick a bin B ∈ S, claim one slot of size ŝ_v in B for v, and add B to ResolvedBins
        else
            Create a new bin B′, add v to B′, and add B′ to ResolvedBins
        end if
    end for
    return ResolvedBins
end procedure
```

Complexity analysis - BFS runs in $O(n)$ time. We have at most $O(n)$ bins each containing at most $O(1/\epsilon)$ items. For each of the n vertices, we can find and filter the available bins in $O(n)$ time. Thus, the overall complexity of the conflict resolution algorithm is $O(n^2)$.

Claim 2. *The solution returned is conflict-free and feasible.*

Proof. By design, the algorithm never adds a vertex to a bin that already contains one of its neighbors, ensuring the solution is conflict-free. Furthermore, since rounding was upward, every item's real size satisfies $s_i \leq \hat{s}_i$. Therefore, the total real size of items placed in any bin from *AbstractedBins* is at most the sum of their abstracted sizes, which is at most 1 (since *AbstractedBins* is a feasible packing of the abstracted sizes). Any new bin created contains only a single item of size at most 1, so these are also feasible. □

Claim 3. *The number of extra bins added is a constant depending only on ϵ.*

Proof. Let $\hat{s}$ be a specific abstracted size. A new bin is opened for an item v of size $\hat{s}$ only if all bins in *AbstractedBins* with an unmatched slot of size $\hat{s}$ are filtered out. Since v has

at most one already-matched neighbor (from the 1-inductive order), it can filter out at most one bin. This implies that when a new bin is opened for size $\hat{s}$, all currently unmatched slots of size $\hat{s}$ must be contained in a single bin.

Because all large items have size at least ϵ, any single bin can contain at most $\lfloor 1/\epsilon \rfloor$ slots. Therefore, at the moment the first new bin is opened for size $\hat{s}$, there are at most $\lfloor 1/\epsilon \rfloor$ unmatched slots of size $\hat{s}$ remaining in total.

Since the total number of items of size $\hat{s}$ equals the total number of slots of size $\hat{s}$ in *AbstractedBins*, every time we place an item of size $\hat{s}$ into a new bin, one slot of size $\hat{s}$ in *AbstractedBins* will permanently remain empty. Consequently, the total number of new bins we can possibly open for size $\hat{s}$ is bounded by the number of available slots for size $\hat{s}$ when the first such new bin is created, which is at most $\lfloor 1/\epsilon \rfloor$.

There are at most $m = \lceil 1/\epsilon^2 \rceil$ distinct abstracted sizes. Thus, the total number of extra bins created across all sizes is at most $\lceil 1/\epsilon^2 \rceil \cdot \lfloor 1/\epsilon \rfloor \leq 1/\epsilon^3 + 1/\epsilon^2$, which is a constant depending only on ϵ. □

Packing small items

Since the conflict graph is a tree, it is bipartite, and we can 2-color the vertices. We will pack the small items greedily into the bins, and those that cannot be packed will be put in new bins according to their color.

Complexity analysis - 2-coloring the tree takes $O(n)$ time using BFS or DFS. The algorithm iterates over at most n small items, and for each item, it checks at most $O(n)$ bins to see if it fits and has no conflicts. Checking for conflicts and capacity constraints can be done efficiently, leading to an overall running time of $O(n^2)$.

Claim 4. *This algorithm yields a feasible packing of the small items.*

Proof. First, the capacity constraint is maintained: an item of size s_i is only added to an existing bin if the sum of sizes including s_i is at most 1. For the new color-specific bins, a fresh bin is opened if the capacity would exceed 1. Thus, no bin exceeds the size limit of 1.

Second, the conflict-free constraint is maintained. When adding to an existing bin, the algorithm explicitly verifies that the bin contains no neighbor of the item. When an item is placed in a new bin, it is grouped only with other items of the same color from the valid 2-coloring of the tree. Since adjacent vertices in a tree (which is bipartite) must have different colors, no two items of the same color can be neighbors. Therefore, the new bins are also strictly conflict-free. Thus, the resulting packing is feasible. □

Putting it all together

Finally, we will describe the APTAS for Bin Packing with Trees.

Proof of correctness

Claim 5. *The solution returned is feasible.*

Proof. By the previous claims, each step of the algorithm maintains feasibility. The ConflictResolution algorithm ensures that all large items are packed into bins such that the

Algorithm 3 Packing small items for Bin Packing with Trees

```
procedure PackSmallItems(Bins, Small, T)
    2-color the vertices of T with colors 1 and 2
    Let Small_1, Small_2 be the small items of colors 1 and 2
    Let NewBins[1] and NewBins[2] be empty bins
    for each c ∈ {1, 2} do
        for each i ∈ Small_c do
            packed ← false
            for each B ∈ Bins do
                if c = 2 and B ∩ Small_1 ≠ ∅ then continue
                if sum of sizes in B + s_i ≤ 1 and B contains no neighbor of i then
                    Add i to B
                    packed ← true
                    break
                end if
            end for
            if not packed then
                if sum of sizes in NewBins[c] + s_i > 1 then
                    Add NewBins[c] to Bins
                    NewBins[c] ← empty bin
                end if
                Add i to NewBins[c]
            end if
        end for
    end for
    Add any non-empty NewBins[1] and NewBins[2] to Bins
    return Bins
end procedure
```

Algorithm 4 APTAS for Bin Packing with Trees

```
procedure BinPackingWithTrees(I, T, ε)
    m = ⌈(2/ε)^2⌉
    Small ← {i | i ∈ I, s_i < ε/2}
    Large ← I \ Small
    sort Large in non-increasing order of size
    Partition Large into m groups G_1, G_2, ..., G_m of sizes as equal as possible
    For each j ∈ [m] and i ∈ G_j, set ŝ_i ← max_{i'∈G_j} s_{i'}
    Bins ← standard APTAS(Large, ε/2)
    Bins ← ConflictResolution(Bins, T)
    Bins ← PackSmallItems(Bins, Small, T)
end procedure
```

sum of their actual sizes is at most 1, and no two adjacent items share a bin. The PackSmallItems algorithm guarantees that the addition of small items also respects both the capacity constraint and the conflict-free constraint. Since every item (both large and small) is placed into some bin without violating these constraints, the final solution is feasible. □

Running time analysis

The algorithm's running time is dominated by the conflict resolution and small item packing steps, both of which take $O(n^2)$ time as analyzed earlier. The initial step of categorizing and rounding the items takes $O(n)$ time. The standard APTAS runs in polynomial time if ϵ is constant. Therefore, the overall running time of the algorithm is polynomial in n for any fixed ϵ.

Approximation ratio analysis

Claim 6. *For a given $\epsilon > 0$, the algorithm returns a packing into at most $(1+\epsilon)OPT + c_\epsilon$ bins.*

Proof. Let OPT be the optimal number of bins for the original instance. The standard APTAS for the large items (with parameter $\epsilon/2$) produces a packing of the abstracted sizes into $m_{abstract} \leq (1+\epsilon/2)\mathrm{OPT} + c$ bins, where c is a constant.

The ConflictResolution algorithm then produces a valid packing for the real large items, opening at most $c_{conflict} \leq 1/(\epsilon/2)^3 + 1/(\epsilon/2)^2$ additional bins, as proven earlier. Let $m_{large} = m_{abstract} + c_{conflict}$ be the total number of bins before packing small items. We have $m_{large} \leq (1+\epsilon/2)\mathrm{OPT} + c'$, where c' is a constant depending only on ϵ.

Next, consider the PackSmallItems phase. Let U be the set of bins from the m_{large} bins that are strictly less than $(1-\epsilon/2)$ full. Any small item i has size $s_i < \epsilon/2$, so it would fit into any bin in U without violating the capacity constraint.

If an item i is placed into a new color-specific bin, it must have been rejected by all bins in U. Since it fits by capacity, it must be rejected due to a conflict. This means i has at least one neighbor in every bin in U.

Let S'_c be the set of small items of color $c \in \{1, 2\}$ that are placed into new bins. Consider S'_1 first. When placing items of $Small_1$, no other small items have been packed yet. Thus, any conflict must be with a large item. Let V_U be the set of large items packed in the bins of U. Since all large items have size at least $\epsilon/2$, each bin in U contains at most $\lfloor 2/\epsilon \rfloor$ items. Thus, $|V_U| \leq |U| \cdot \lfloor 2/\epsilon \rfloor$.

The edges between S'_1 and V_U form a bipartite subgraph of the tree T, which must be a forest. In a forest, the number of edges is strictly less than the number of vertices. Since every item in S'_1 has at least one large neighbor in every bin in U, there are at least $|S'_1| \cdot |U|$ edges. Therefore:

$$|S'_1| \cdot |U| \leq |S'_1| + |V_U| - 1 \implies |S'_1|(|U| - 1) < |U| \cdot \lfloor 2/\epsilon \rfloor$$

The exact same logic applies to S'_2: because the algorithm explicitly forbids packing $Small_2$ into bins containing $Small_1$, any conflicts that force an item in $Small_2$ into a new bin must also be with large items in V_U. Thus, $|S'_2|(|U| - 1) < |U| \cdot \lfloor 2/\epsilon \rfloor$.

We consider two cases for $|U|$:

- **Case 1:** $|U| \geq 2$**.** Then for each color c, $|S'_c| < \frac{|U|}{|U|-1}\lfloor 2/\epsilon \rfloor \leq 2\lfloor 2/\epsilon \rfloor \leq 4/\epsilon$. This means at most $8/\epsilon$ small items are placed into new bins in total. Since each new bin contains items of the same color, and we only open a new bin when capacity is exceeded, these items will open at most $O(1/\epsilon)$ new bins. The total number of bins is $m_{large} + O(1/\epsilon) \leq (1+\epsilon/2)\mathrm{OPT} + c_\epsilon \leq (1+\epsilon)\mathrm{OPT} + c_\epsilon$.

- **Case 2:** $|U| \leq 1$**.** This means all but at most one of the m_{large} bins are filled to at least $1-\epsilon/2$. Let m_{total} be the final number of bins. All newly opened bins (except possibly the last one for each color) are also filled to at least $1-\epsilon/2$. Thus, at least $m_{total}-3$ bins have a sum of sizes greater than $1-\epsilon/2$. The total size of all items in the instance, which is a lower bound on OPT, is at least $(m_{total}-3)(1-\epsilon/2)$. Therefore, $m_{total} \leq \frac{\mathrm{OPT}}{1-\epsilon/2} + 3$. For $\epsilon \leq 1$, $\frac{1}{1-\epsilon/2} \leq 1+\epsilon$. So $m_{total} \leq (1+\epsilon)\mathrm{OPT} + 3$.

In both cases, the final number of bins is bounded by $(1+\epsilon)\mathrm{OPT} + c_\epsilon$ for some constant c_ϵ depending only on ϵ, satisfying the definition of an APTAS. $\square$

Problem 5: My own question

Propose a question related to the material covered in the course and include a complete and formal solution. The level is expected to be similar to the level of other questions in this assignment. Extremely well written and elegant question may receive a bonus.

The question - The taxi problem

Let $G = (V, E)$ be a complete graph. Let $t : V \times V \to \mathbb{R}_+$ be a metric representing the travel time between any two vertices. We are given an integer $B \geq 1$ representing the number of available taxis, and a set of n passengers. Each passenger i is defined by a tuple $(s_i, d_i) \in V \times V$, where s_i is their source vertex and d_i is their destination vertex.

All B taxis have infinite passenger capacity, travel at a constant speed of 1, and begin their routes at time 0 from any chosen starting vertices in V. Once a passenger is picked up by a taxi at their source s_i, they must remain in that same taxi until they are dropped off at their destination d_i (i.e., transfers between taxis are not allowed).

A valid schedule defines the starting vertex and the sequence of visited vertices for each taxi over time such that every passenger i is successfully transported from s_i to d_i. Let C_i denote the completion time (arrival time) of passenger i at their destination d_i.

find valid schedules for all B taxis that minimize the maximum completion time among all passengers: $\max_i C_i$.

Show that this problem is NP-hard, and show a k-approximation algorithm for the problem for $B = 1$.

NP-hardness

Theorem 1. *The taxi problem is NP-hard, even for a single taxi ($B = 1$).*

Proof. We show that the taxi problem is NP-hard even for a single taxi ($B = 1$), by reduction from the **Hamiltonian Path Problem**.

Reduction. Given an instance $H = (V_H, E_H)$ of Hamiltonian Path, we construct a taxi instance as follows.

- **Vertices.** Let $V = V_H \cup \{d^*\}$, where d^* is a fresh depot vertex not in V_H.
- **Metric.** Define the travel time t as the shortest-path distance in the complete graph where:
  - For $u, v \in V_H$: if $(u, v) \in E_H$, set $w(u, v) = 1$; otherwise $w(u, v) = 2$.
  - For $v \in V_H$: set $w(v, d^*) = m^2$.

  Then define $t(u, v)$ as the shortest-path distance under w. Since shortest-path distances satisfy the triangle inequality, t is a valid metric, and it is computable in polynomial time.

  Note that for $u, v \in V_H$: $t(u, v) = 1$ if $(u, v) \in E_H$, and $t(u, v) = 2$ otherwise (the direct weight of 2 is at most any two-hop path of cost ≥ 2).

- **Taxis.** $B = 1$.
- **Passengers.** For each vertex $v_i \in V_H$, create a passenger with source $s_i = v_i$ and destination $d_i = d^*$.
- **Threshold.** $T = (m-1) + m^2$.

Correctness.

In any valid schedule, the single taxi must visit every source $v_i \in V_H$ to pick up all m passengers, and then reach d^* to drop them off. Since the taxi has infinite capacity, the optimal strategy is to first tour all of V_H (picking up everyone), and then travel to d^* (dropping everyone off). The makespan equals

$$C_{\max} = (\text{length of path through all vertices of } V_H) + t(v_{\text{last}}, d^*),$$

where v_{last} is the last vertex of V_H visited. Since $t(v, d^*) = m^2$ for every $v \in V_H$, this simplifies to

$$C_{\max} = (\text{length of path through all vertices of } V_H) + m^2.$$

($\Rightarrow$) If H has a Hamiltonian path $v_{\pi(1)}, v_{\pi(2)}, \ldots, v_{\pi(m)}$, the taxi starts at $v_{\pi(1)}$ and follows this path. Each edge has t-cost 1 (since consecutive vertices are adjacent in H), so the path has total length $m-1$. The end time is $(m-1) + m^2 = T$.

($\Leftarrow$) Suppose there is a valid schedule with $\leq T = (m-1) + m^2$. The taxi visits all m vertices of V_H in some order, traversing $m-1$ edges, then travels to d^* at cost m^2. The total path length through V_H must be $\leq m-1$. Since each step between consecutive visited vertices costs at least 1 (as $t(u,v) \geq 1$ for all $u \neq v$ in V_H), and there are exactly $m-1$ steps, each step must cost exactly 1. By construction, $t(u,v) = 1$ if and only if $(u,v) \in E_H$. Therefore the sequence of visited vertices forms a Hamiltonian path in H.

The reduction runs in polynomial time (constructing the metric takes $O(m^2)$ time). Since the Hamiltonian Path Problem is NP-complete, the taxi problem (even with $B = 1$) is NP-hard. □

K approximation algorithm for the taxi problem

Algorithm. the idea for greedy algorithm for the general case is the following. the time for a taxi with the passengers list $\{a_1, \ldots, a_m\}$ is the TSP approximation of the path for the sources, then the shortest path from the last source to the first destination, then the TSP approximation of the path for the destinations. at each iteration, we assign the next passenger such that the increase in the completion time of all the taxis is minimal. we repeat until all passengers are assigned.

here is the algorithm for the single taxi case ($B = 1$):

Complexity. The algorithm runs in polynomial time, since Christofides' algorithm runs in polynomial time, and finding the minimum $t(s_{last}, d_i)$ takes $O(n)$ time.

Approximation Analysis.

Algorithm 5 4-Approximation for Single Taxi ($B = 1$)

```
procedure SingleTaxiSchedule(G, t, {(s_1, d_1), ..., (s_n, d_n)})
    S ← {s_1, ..., s_n}
    D ← {d_1, ..., d_n}
    Phase 1: Collect all passengers
    Path_S ← ChristofidesPath(S, t)
    s_last ← last vertex in Path_S
    Phase 2: Transition
    d_first ← Find d_i ∈ D that minimizes t(s_last, d_i)
    Phase 3: Deliver all passengers
    Path_D ← ChristofidesPath(D, t, start = d_first)
    return Path_S ⊕ Path_D                    ▷ Concatenate the two paths
end procedure
```

Claim 7. *The greedy algorithm is a 4-approximation algorithm for the single taxi problem ($B = 1$).*

Proof. Let OPT be the optimal maximum completion time (makespan) for a single taxi to deliver all passengers. Let $S = \{s_1, \ldots, s_n\}$ be the set of all source vertices and $D = \{d_1, \ldots, d_n\}$ be the set of all destination vertices.

1. Lower Bounds on OPT:

Any valid optimal schedule must visit all vertices in S to pick up the passengers, and it must visit all vertices in D to drop them off. Because the travel times t form a metric (satisfying the triangle inequality), shortcutting any intermediate vertices can only decrease the total path length. Therefore, we can establish the following lower bounds:

- **Lower Bounds 1 & 2 (Sources and Destinations):** The optimal schedule contains a path that visits all vertices in S and all vertices in D. Thus, the shortest paths covering all sources and destinations, denoted $OPT_{path}(S)$ and $OPT_{path}(D)$, are both lower bounds on OPT:

$$OPT \geq OPT_{path}(S) \quad \text{and} \quad OPT \geq OPT_{path}(D)$$

- **Lower Bound 3 (Max Edge):** The optimal schedule must visit every node in $S \cup D$. By the triangle inequality, the distance between *any* two nodes in $S \cup D$ is at most the length of the optimal path. We have: $OPT \geq t(s_{last}, d_{first})$

2. Upper Bound on the Algorithm's Cost:

The proposed algorithm constructs a schedule in three sequential phases:

1. A TSP path covering S.
2. A single transit edge from s_{last} to d_{first}.
3. A TSP path covering D.

Let ALG be the total completion time of this algorithm. The cost is the sum of the three phases:

$$ALG = Cost(Path_S) + t(s_{last}, d_{first}) + Cost(Path_D)$$

We are assuming the use of Christofides' algorithm adapted for the Shortest Hamiltonian Path problem with free endpoints. This algorithm guarantees a 1.5-approximation for the optimal path. Therefore:

- $Cost(Path_S) \leq 1.5 \cdot OPT_{path}(S)$
- $Cost(Path_D) \leq 1.5 \cdot OPT_{path}(D)$

3. Combining Bounds:
Substitute the approximations and the lower bounds into the algorithm's cost equation:

$$ALG \leq 1.5 \cdot OPT_{path}(S) + t(s_{last}, d_{first}) + 1.5 \cdot OPT_{path}(D)$$

Using our lower bounds ($OPT_{path}(S) \leq OPT$, $t(s_{last}, d_{first}) \leq OPT$, and $OPT_{path}(D) \leq OPT$):

$$ALG \leq 1.5 \cdot OPT + 1 \cdot OPT + 1.5 \cdot OPT$$

$$ALG \leq 4 \cdot OPT$$

Thus, the proposed greedy algorithm is a **4-approximation algorithm** for the single taxi problem. □

Problem 6: Linear Programming on Multidimensional Knapsack

In the Multidimensional Knapsack problem we are given a d-dimensional knapsack, for some $d \geq 2$, having capacity b_i in the ith dimension, for $1 \leq i \leq d$. Also, there is a set of items $I = \{1, \ldots, n\}$. The size of item j in dimension i is $s_{ij} > 0$, $1 \leq j \leq n$, $1 \leq i \leq d$. The profit from packing item j is $p_j > 0$. A solution is a subset of items $A \subseteq I$, and the profit of A is $P(A) = \sum_{j \in A} p_j$. A solution A is feasible if $\sum_{j \in A} s_{ij} \leq b_i$ for all $1 \leq i \leq d$. We assume below that d is a fixed constant. Consider the following algorithm for d-dimensional Knapsack. Let $\epsilon > 0$ and $h = \min(n, \lceil d(1+\epsilon)/\epsilon \rceil)$. Given a selected subset of items $S \subseteq I$, the objective of the S-problem is to find a feasible packing of items in the knapsack which maximizes the profit, subject to the following constraints: (i) All items in S are packed. (ii) Among the items in $I \setminus S$ only items j with profit $p_j \leq \min_{\ell \in S} p_\ell$ can be added to the solution. Denote this set of items by Good(S). Given the selected subset of items S, we denote by LP (S) a linear programming relaxation of the S-problem. Let x_j be an indicator for the selection of item $j \in I$ for the solution. In the linear programming relaxation $LP(S)$, $0 \leq x_j \leq 1$. Let $x^B = (x_1^B, \ldots, x_n^B)$ be a basic solution for $LP(S)$.

The Algorithm: For any $S \subseteq I$ such that $|S| \leq h$, solve optimally $LP(S)$ and round down x^B. Choose the solution which maximizes the profit. (a) (2 pt.) Formulate the linear program $LP(S)$. (b) (5 pt.) Denote by x_j^I the integer variable obtained after rounding the basic variable x_j^B. Let F be the subset of items j for which x_j^B is fractional, i.e., $x_j^I < x_j^B$. Show that $|F| \leq d$.

Formulation of LP(S)

The linear programming relaxation $LP(S)$ can be formulated as follows:

Maximize:

$$\sum_{j \in I} p_j x_j$$

Subject to:

$$\begin{aligned}
\sum_{j \in I} s_{ij} x_j &\leq b_i && \forall i \in \{1, \ldots, d\} \\
x_j &= 1 && \forall j \in S \\
x_j &= 0 && \forall j \in I \setminus (S \cup \text{Good}(S)) \\
0 \leq x_j &\leq 1 && \forall j \in \text{Good}(S)
\end{aligned}$$

Bounding the number of fractional variables

Let x^B be a basic feasible solution to $LP(S)$. There are n variables in the linear program. In a basic feasible solution, there must be exactly n linearly independent tight (active) constraints.

From the formulation of $LP(S)$, we have the following constraints that can be tight:

1. $x_j = 1$ for $j \in S$ (exactly $|S|$ constraints)

2. $x_j = 0$ for $j \in I \setminus (S \cup \text{Good}(S))$ (exactly $n - |S| - |\text{Good}(S)|$ constraints)

3. $x_j = 0$ or $x_j = 1$ for $j \in \text{Good}(S)$

4. $\sum_{j \in I} s_{ij} x_j \leq b_i$ for $i \in \{1, \ldots, d\}$ (at most d constraints)

Let k be the number of variables x_j for $j \in \text{Good}(S)$ that are tight at their bounds (i.e., $x_j = 0$ or $x_j = 1$). Since a basic feasible solution is determined by n linearly independent tight constraints, we must have:

$$|S| + (n - |S| - |\text{Good}(S)|) + k + (\text{number of tight capacity constraints}) \geq n$$

This simplifies to:

$$n - |\text{Good}(S)| + k + (\text{number of tight capacity constraints}) \geq n$$

$$k \geq |\text{Good}(S)| - (\text{number of tight capacity constraints})$$

Since there are only d capacity constraints in total, the number of tight capacity constraints is at most d. Thus:

$$k \geq |\text{Good}(S)| - d$$

This means that out of the $|\text{Good}(S)|$ variables in $\text{Good}(S)$, at least $|\text{Good}(S)| - d$ of them are fixed to 0 or 1. The remaining variables are the only ones that can be strictly fractional ($0 < x_j < 1$). Therefore, the number of fractional variables is at most:

$$|F| = |\text{Good}(S)| - k \leq |\text{Good}(S)| - (|\text{Good}(S)| - d) = d$$

Therefore, $|F| \leq d$.